



Destilación y Resolución con Skills Neuro-Simbólicos

> Fase 1: Síntesis de Libro a Skill | Fase 2: Razonamiento y Oráculo SymPy

Ficha Técnica del Documento de Estudio

Fecha:	Septiembre 2026	Arquitectura:	3 Capas Deterministas
Coste E2E (452 págs):	~\$0.75 USD (Fase 1 + Fase 2)	Oráculo Simbólico:	SymPy 1.14.0 (Sandbox AST)
Modelos Tier:	DeepSeek V4 Pro / Gemini Flash	Tasa Validación:	33/33 Bloques SymPy (100 %)

Índice

PARTE I: Fase 1 — De un Libro PDF a un Skill Estructurado	2
1. Qué Resuelve la Fase 1	2
2. Arquitectura de 3 Capas (Fase 1)	3
3. El Orquestador flujo_libro_a_skill.py	3
3.1. Estrategia Determinista de Chunking	4
4. Motor de Síntesis sintetizar_skill.py	4
4.1. Dos Fases Internas de Procesamiento	4
4.2. Saneamiento Determinista del JSON y Frontmatter	4
5. Validación Neuro-Simbólica y Oráculo SymPy	5
6. Resultados Experimentales E2E	5
 PARTE II: Fase 2 — Resolver Problemas con un Skill Estructurado	 6
7. Qué Resuelve la Fase 2	6
8. Arquitectura de 3 Capas (Fase 2)	7
9. Motor de Ejecución resolver_skill.py	7
9.1. 1. Retrieval Semántico Multi-Skill por Embeddings (0 créditos)	7

9.2.	2. Formulador LLM y Estructura de Salida	8
9.3.	3. Oráculo SymPy y Bucle de Reflexión	8
10.	Validación Experimental Fase 2	8
10.1.	Validación Experimental de Confianza del Retrieval (2026-09-03)	9
 PARTE III: Operatoria, Costes y Síntesis Global		10
11.	Guía de Comandos Operativos	10
12.	Comparativa y Síntesis Global	12

PARTE I: Fase 1 — De un Libro PDF a un Skill Estructurado

1 Qué Resuelve la Fase 1: Destilación Neuro-Simbólica

Un modelo de lenguaje (LLM) que simplemente “responde preguntas sobre un libro técnico” tiende a alucinar constantes, inventar ecuaciones y saltarse hipótesis fundamentales. La **Fase 1** transforma un documento técnico en PDF en un **skill estructurado** que permite al agente razonar formalmente:

- **Teoría y definiciones rigurosas:** Sin resúmenes superficiales; preservando supuestos y rango de validez.
- **Metodologías de resolución paso a paso:** Plantillas algorítmicas que replican el método del autor.
- **Límites de aplicabilidad y prerequisites:** Condiciones exactas de borde (*cuándo NO aplicar*).
- **Fórmulas ejecutables en SymPy:** Cada fórmula matemática se representa como código ejecutable, permitiendo validación formal determinista con 0 alucinaciones.

Pipeline E2E de Destilación de la Fase 1

```

1  [Libro PDF] --> extraer_libro_pdf.py --> texto_completo.txt --> enrutador.
    py
2
3                                     |
4                                     (destilacion chunk a chunk |
5                                     -> corpus)
6                                     (ensamblaje 7 archivos) |
7                                     v
8                                     SKILL.md + references/ (
9                                     formulas,
10                                    metodologias, limites,
11                                    prerequisitos,
12                                    tablas, glosario)
                                     v
[validar_skill_formulas.py -> gate
 instalacion]
[generar_latex_skill.py -> reporte
 tecnico PDF]
[instalar_skill.py -> ~/.config/
 opencode/skills]
```

2 Arquitectura de 3 Capas de la Fase 1

Siguiendo el principio de diseño del proyecto, toda la funcionalidad se divide estrictamente en 3 capas desacopladas:

Capa	Archivo / Artefacto	Responsabilidad y Rol
Directiva	directives/libro_a_skill.yaml	SOP determinista: 8 pasos secuenciales, 13 edge cases catalogados y política de reintentos.
Orquestación	flujo_libro_a_skill.py	Entrevista al usuario, orquestación de scripts, manejo de estados en .tmp/ y gate de instalación.
Ejecución	execution/sintetizar_skill.py execution/extraer_libro_pdf.py execution/validar_skill_formulas.py execution/generar_latex_skill.py execution/instalar_skill.py	Scripts unitarios deterministas con entrada/salida estricta. Cero lógica de negocio en prompts del chat.

Tabla 1: Matriz de responsabilidades de la arquitectura de 3 capas en la Fase 1.

3 El Orquestador `flujo_libro_a_skill.py`

El orquestador ejecuta una secuencia rigurosa de 8 pasos:

1. **Paso 1 — Entrevista interactiva / CLI:** Captura nombre (*snake_case*), tema, idioma y alcance. Persiste estado en `.tmp/entrevista_skill_<nombre>.json`.
2. **Paso 2 — Extracción determinista (0 créditos):** Ejecuta `extraer_libro_pdf.py`. En el test E2E real extrajo 452 páginas, 1.6M caracteres y ~400K tokens.
3. **Paso 3 — Enrutamiento inteligente:** Invoca `execution/enrutador.py -task contexto_masivo` para seleccionar deterministamente el tier **deepseek** (`deepseek/deepseek-v4-pro`).
4. **Paso 4 — Síntesis y destilación (créditos):** Invoca `sintetizar_skill.py` produciendo los 7 archivos estructurados.
5. **Paso 5 — Validación neuro-simbólica (0 créditos):** Ejecuta `validar_skill_formulas.py`. Si hay fórmulas inválidas, dispara bucle de reflexión hasta ≤ 3 veces.
6. **Paso 6 — Reporte LaTeX infográfico (0 créditos):** Genera reporte PDF en `docs/SKILL/<nombre>/.pdf`.
7. **Paso 7 — Instalación global condicionada (0 créditos):** Copia a `~/config/opencode/skills/<nombre>/.py`.
8. **Paso 8 — Alerta sonora y telemetría:** Notifica al usuario con `alert_user.py` avisando de reiniciar el entorno.

⚠ Punto crítico de diseño

Gate de Instalación Seguro (Implementado 2026-09-01): Si la validación del Paso 5 arroja bloques simbólicos rotos, la instalación en el directorio global se **omite automáticamente** emitiendo un warning claro, pero **no se aborta el flujo**. De esta forma se genera siempre el reporte LaTeX con los diagnósticos sin contaminar el entorno de producción con skills defectuosos.

3.1 Estrategia Determinista de Chunking

La función `_repartir_chunks(texto, max_tokens)` divide el texto por párrafos (`\n\n`) hasta alcanzar `max_chars = max_tokens * 4`. Párrafos más extensos que el límite se dividen a nivel de página. Con el default de 16000 tokens por chunk, un libro de 400K tokens se particiona limpiamente en ≈ 27 chunks.

4 Motor de Síntesis: `sintetizar_skill.py`

4.1 Dos Fases Internas de Procesamiento

- **Fase A — Destilación Chunk a Chunk (DeepSeek):** Cada chunk se envía con un prompt especializado que produce un resumen denso en markdown con definiciones, variables y ecuaciones en notación $\$. \dots \$$. Fragmentos no técnicos devuelven `[sin contenido relevante]` y se filtran. El resultado se guarda en `.tmp/skill_<nombre>_destilado.txt` como **corpus destilado**.
- **Fase B — Ensamblaje Estructurado (DeepSeek):** Genera un JSON único con `SKILL.md` y exactamente las 6 referencias formales obligatorias:
 1. `formulas.md`: Ecuaciones con SymPy, dominio y notas de validez.
 2. `metodologias.md`: Plantillas numeradas paso a paso del procedimiento de cálculo.
 3. `limites_aplicabilidad.md`: Supuestos de borde (*cuándo NO aplicar*).
 4. `prerrequisitos.md`: Grafo de dependencias de conceptos y herramientas.
 5. `tablas.md` y `glosario.md`: Parámetros de componentes y definiciones formales.

4.2 Saneo Determinista del JSON y Frontmatter

Debido a imperfecciones típicas de LLMs al emitir JSONs de gran tamaño, se aplican algoritmos deterministas de reparación:

- `_sanear_json`: Corrige *trailing commas* (`(, } → }`) y escapa barras invertidas de LaTeX crudas (`\Omega → \\Omega`) sin alterar caracteres de escape válidos.

- `_find_balanced_json`: Extractor determinista mediante escaneo de llaves balanceadas que respeta cadenas y escapes, evitando fallos de expresiones regulares no codiciosas.
- `_normalizar_frontmatter + _yaml_q`: Re-emite escalares YAML entre comillas dobles, evitando que caracteres especiales como dos puntos (:) rompan el encabezado del skill.

💡 Concepto clave

Presupuesto de Ensamblaje y Solución de Truncamiento: En libros completos, las 6 referencias estructuradas superan el límite de salida estándar de 8192 tokens. Se implementó el parámetro `-estructura-max-tokens 32768` para tier DeepSeek y el flag `-usar-corpus`, el cual permite re-ensamblar un skill existente sin volver a pagar la destilación chunk a chunk (ahorro de ~30 min y ~\$0.70 USD).

5 Validación Neuro-Simbólica: `validar_skill_formulas.py`

El validador escanea todos los bloques Python en `SKILL.md` y `references/*.md` sin consumir créditos de API:

1. **Análisis de Sintaxis:** Parseo de Abstract Syntax Tree (`ast.parse`).
2. **Escaneo de Seguridad AST:** Bloquea llamadas inseguras (`os`, `subprocess`, `eval`, `exec`, `dunders`).
3. **Ejecución en Sandbox Aislado:** Ejecución controlada con `subprocess.run([sys.executable, -c, src], timeout=5 s)` en el validador de la Fase 1, y `timeout=30 s` en el oráculo de la Fase 2.
4. **Verificación de Estructuras Obligatorias:** Comprueba presencia de metodologías, límites y prerequisites.

6 Resultados Experimentales en Libro Completo (Fase 1)

Datos obtenidos en la prueba de validación E2E sobre el libro “*Circuitos y Dispositivos Electrónicos*” (Lluís Prat Viñas):

Métrica de Evaluación	Valor Observado en Ejecución Real
Volumen de entrada	452 páginas / 1.6M caracteres / ~400K tokens
Modelo y Tier de enrutamiento	Tier deepseek (deepseek/deepseek-v4-pro)
Bloques SymPy extraídos y validados	33/33 Bloques OK (100 % de éxito, 0 errores)
Re-síntesis por reflexión requerida	0 iteraciones adicionales (éxito en pasada inicial)
Artefactos estructurados producidos	7 archivos (SKILL.md + 6 referencias técnicas)
Ubicación de reportes	docs/SKILL/circuitos_dispositivos_electronicos/
Instalación final	~/.config/opencode/skills/circuitos_dispositivos_electronicos
Coste total de la fase	~\$0.75 USD

Tabla 2: Resultados del proceso de destilación neuro-simbólica E2E.

PARTE II: Fase 2 — Resolver Problemas con un Skill Estructurado

7 Qué Resuelve la Fase 2: Patrón Neuro-Simbólico

Habiendo creado los skills en la Fase 1, la **Fase 2** los utiliza para resolver problemas cuantitativos de ingeniería. Soporta **múltiples skills del mismo dominio** (Opción C): el resolver hace retrieval sobre *todos* ellos y combina los resultados, ampliando el campo de conocimiento sin re-destilar. El principio fundamental es:

El LLM jamás calcula mentalmente: razona cualitativamente y formula un programa simbólico; el oráculo determinista ejecuta y valida.

Flujo de Razonamiento, Formulación y Validación Simbólica

```
1  [Problema + N Skills del dominio]
2      |
3      v
4  [Retrieval Embeddings (multi-skill, 0 credits)]
5      - retrieval sobre TODOS los skills
6      - combina y corta al top-k global
7      - confianza_retrieval + mejor score por skill
8      |
9      v
10 [LLM: Formulador] --> [Oraculo SymPy]
11   | (analisis +      |
12   |  codigo SymPy)   |
13 +---- feedback -----+
14   |
15   [Reflexion Local <=3]
```

8 Arquitectura de 3 Capas de la Fase 2

Capa	Archivo / Artefacto	Responsabilidad y Rol
Directiva	directives/resolver_skill.yaml	SOP: validación de entradas, pasos, 8 edge cases y presupuesto de retry.
Orquestación	flujo_resolver_skill.py	Valida el/los problema(s) y skills, consulta al enrutador determinista, invoca el solver y reporta resultados.
Ejecución	execution/resolver_skill.py	Motor de retrieval multi-skill por similitud coseno, formulador LLM, sandbox del oráculo y reflexión.

Tabla 3: Arquitectura de 3 capas para la resolución neuro-simbólica de problemas.

9 Motor de Ejecución: resolver_skill.py

9.1 1. Retrieval Semántico Multi-Skill por Embeddings (0 créditos)

- Divide SKILL.md y referencias en secciones respetando encabezados markdown (# ... ###).
- **Multi-skill (Opción C):** -skill acepta varios directorios de skills afines. El retrieval se ejecuta sobre *todos* (cada uno con su caché de embeddings) y los resultados se **combinan con cuotas por skill**:
 - **Reserva obligatoria:** se incluye siempre la mejor sección de cada skill, para que un skill de dominio con pocas secciones de score alto nunca quede desplazado por un skill lateral con muchos chunks de score medio (evita dilución).
 - **Cuotas equitativas:** el resto del top- k se completa repartiendo lo más parejo posible entre los skills (round-robin por score desc), sin que un skill lateral sature el contexto.
 - **Deduplicación:** la misma sección presente en varios skills se conserva una sola vez (la de mayor score).
 - Cada sección se etiqueta con su fuente (skill de origen) y se reporta el **mejor score por skill** (confianza_retrieval_por_skill, calculado sobre todas sus secciones recuperadas, no solo las entregadas al LLM).
- Genera representaciones vectoriales mediante el modelo local paraphrase-multilingual-MiniLM-L12-v2.
- Aplica similitud coseno seleccionando el top- k de secciones con score $>$ min_score (default $k = 6$, umbral 0.05).
- **Confianza del fundamento** (confianza_retrieval): clasifica el **mejor score combinado** en alta (≥ 0.55), media ($\geq$ alerta_score, default 0.35) o baja (< 0.35). Umbrales calibrados con evidencia medida (ver §10.2). Reporta además mejor_skill (el más afín) y skills_sin_secciones.

- Implementa caché determinista en `.tmp/resolver_skill/<skill>/` con `secciones.json` y `embeddings.npz` basados en hashes **SHA-256** del contenido.
- **Ampliación del conocimiento:** si un problema es difícil y la confianza queda baja, ese es el indicador de que hace falta añadir más PDFs/skills del dominio (cada nuevo libro de la Fase 1 se usa como skill adicional sin re-destilar).

9.2 2. Formulador LLM y Estructura de Salida

El modelo recibe las secciones recuperadas y está obligado a responder con un JSON estricto de 3 claves:

</> Contrato de Datos del Formulador

```
1 {
2   "analisis": "Razonamiento paso a paso citando la Metodología y limites",
3   "codigo_sympy": "from sympy import *\n# Variables, ecuaciones y print\n    formal",
4   "resultado_esperado": "Valor numerico o simbolico esperado (prediccion)"
5 }
```

9.3 3. Oráculo SymPy y Bucle de Reflexión

- **Seguridad previa:** Inspección AST con `_escaneo_seguridad`.
- **Sandbox:** Ejecución aislada con límite de tiempo (`timeout=30s`).
- **Bucle de Reflexión (≤ 3):** Ante fallos de ejecución (`NameError`, `TypeError`, etc.), se inyecta el `stderr` real del compilador al LLM para corregir el código sin alterar la estrategia física de resolución.

10 Validación Experimental en Circuito BJT Real

Se evaluó la resolución del punto de operación Q de un transistor BJT en configuración emisor común con divisor de tensión en base ($V_{CC} = 12\text{ V}$, $R_1 = 100\text{ k}\Omega$, $R_2 = 50\text{ k}\Omega$, $R_E = 1\text{ k}\Omega$, $R_C = 2\text{ k}\Omega$, $\beta = 100$):

✓ Resultados de la Ejecución Experimental de Fase 2

Sección top recuperada:	“Metodología 7: Análisis BJT en DC” (Score: 0.680)
Razonamiento del LLM:	Citó explícitamente la metodología del skill y verificó hipótesis activas de unión.
Salida del Oráculo SymPy:	$I_{CQ} = 2.4566 \text{ mA}$, $V_{CEQ} = 4.6057 \text{ V}$ (Exactitud 100 %)
Reflexiones necesarias:	0 rondas (Resuelto y validado a la primera pasada)
Coste de inferencia:	~\$0.005 USD (utilizando Tier Flash para validación rápida)

10.1 Validación Experimental de Confianza del Retrieval (2026-09-03)

Para que el sistema distinga robustamente *problemas dentro de alcance* de los que **no están reseñados en el PDF**, se calibaron los umbrales de confianza con evidencia real medida sobre el skill del libro (modelo paraphrase-multilingual-MiniLM-L12-v2):

Tipo de problema	Mejor score coseno medido
Dentro de alcance (BJT punto Q, Ley de Ohm, Thévenin, Kirchhoff)	0.45 – 0.75
Fuera de alcance (química orgánica, poesía griega)	~0.25

Tabla 4: Separación empírica en/out-of-scope que sustenta la calibración de umbrales.

Con los defaults iniciales (alerta_score=0.20) la confianza baja jamás se disparaba en la práctica (incluso un problema ajeno daba ~0.25). Por ello se ****recalibró**** con la separación medida: umbral de confianza débil alerta_score = 0.35 y tope de confianza alta en 0.55.

🛡 Detección de Fundamento Débil y Ampliación Multi-Skill (Medición Real, 0 créditos)

Problema fuera de alcance (química)	best score 0.2648 → confianza baja
Problema dentro de alcance (BJT)	best score 0.7092 → confianza alta
Abortar por fundamento débil (-abortar-debil)	exit 3 antes del LLM (0 créditos)
Aviso al usuario	“Confianza del retrieval BAJA (mejor score X)”
Formulador con fundamento débil	inyecta “no cubierto por el libro” en vez de forzar fórmula
Multi-skill (Electrónica + Física)	mejor score por skill (p. ej. 0.68 / 0.35); mejor_skill = el más afín
Confianza baja en problema difícil	señal de que conviene añadir más PDFs/skills del dominio

El campo `confianza_retrieval` {nivel, mejor_score, alerta, alerta_score, skills_consultados, skills_sin_secciones, mejor_skill, por_skill} se incluye en el JSON de salida, permitiendo auditar la fiabilidad de la respuesta y ****detener el flujo (exit 3) sin gastar créditos**** cuando el problema cae fuera del alcance del knowledge de los skills. En modo multi-skill la confianza se calcula sobre el **mejor score combinado** de todos los skills consultados.

PARTE III: Operatoria, Costes y Síntesis Global

11 >_ Guía de Comandos Operativos

>_ Comandos para la Fase 1 (Creación del Skill)

```
1 # 1. Flujo completo de sintesis con validacion y reporte
2 python3 flujo_libro_a_skill.py --pdf "libro.pdf" --nombre mi_skill \
3     --tema "Electronica Analogica" --idioma es --sobrescribir --reflexion 1
4
5 # 2. Modo Dry-Run (solo genera y valida, sin instalar globalmente)
6 python3 flujo_libro_a_skill.py --pdf "libro.pdf" --nombre mi_skill \
7     --tema "Electronica Analogica" --dry-run
8
9 # 3. Re-ensamblar reutilizando el corpus ya destilado (Ahorro de ~90%
10     credits)
11 python3 flujo_libro_a_skill.py --pdf "libro.pdf" --nombre mi_skill \
12     --tema "Electronica Analogica" --sobrescribir --usar-corpus --
13     estructura-max-tokens 32768
14
15 # 4. Validar formulas simbolicas de forma 100% local (0 credits)
16 python3 execution/validar_skill_formulas.py --skill ~/.config/opencode/
17     skills/mi_skill
```

>_ Comandos para la Fase 2 (Resolución de Problemas)

```

1  # 1. Resolver problema en modo produccion (calculo_formal -> Opus)
2  python3 flujo_resolver_skill.py --problema "Calcular el punto Q de un BJT
   ... " \
3      --skill ~/.config/opencode/skills/circuitos_dispositivos_electronicos
4
5  # 2. Test rapido y economico con override a modelo Flash (~$0.005)
6  python3 flujo_resolver_skill.py --problema "Calcular el punto Q de un BJT
   ... " \
7      --skill ~/.config/opencode/skills/circuitos_dispositivos_electronicos \
8      --modelo google/gemini-3.7-flash --max-reflexion 1
9
10 # 3. Inspeccionar unicamente las secciones relevantes recuperadas (0
    credits)
11 python3 execution/resolver_skill.py --problema "Calcular el punto Q de un
    BJT..." \
12     --skill ~/.config/opencode/skills/circuitos_dispositivos_electronicos
    --solo-retrieval
13
14 # 4. Abortar si el retrieval tiene fundamento debil (confianza baja) ->
    exit 3
15 python3 flujo_resolver_skill.py --problema "..." \
16     --skill ~/.config/opencode/skills/circuitos_dispositivos_electronicos
    --abortar-debil
17
18 # 5. Ajustar el umbral de confianza debil (default 0.35; mas estricto = mas
    alto)
19 python3 flujo_resolver_skill.py --problema "..." --skill <dir> --alerta-
    score 0.40
20
21 # 6. AMPLIAR el campo de conocimiento: varios skills del mismo dominio
22 #   (Electronica + Fisica). Retrieval combinado, top-k global, 0 credits
    extra.
23 python3 flujo_resolver_skill.py --problema "Calcular el punto Q de un BJT
    ..." \
24     --skill ~/.config/opencode/skills/circuitos_dispositivos_electronicos \
25     ~/.config/opencode/skills/electronica_2 ~/.config/opencode/skills/
    fisica_electronica
26
27 # 7. Solo retrieval multi-skill: ver que aporta cada skill (0 credits)
28 python3 execution/resolver_skill.py --skill <dir1> <dir2> \
29     --problema "..." --solo-retrieval

```

12 Cuadro Comparativo: Fase 1 vs Fase 2

Eje de Comparación	Fase 1: Síntesis de Skill	Fase 2: Resolución con Skill
Objetivo Primario	Extraer y destilar el conocimiento de un libro en un skill formal.	Aplicar el conocimiento estructurado para resolver problemas.
Entrada	Archivo PDF técnico (centenares de páginas).	Enunciado del problema de ingeniería.
Salida	SKILL.md + 6 archivos en references/.	JSON con análisis, código SymPy y resultado verificado.
Rol del LLM	Destilador y estructurador formal de contenido.	Razonador cualitativo y formulador simbólico.
Oráculo Determinista	Validador estático/ejecutable de bloques SymPy.	Ejecutor dinámico del cálculo y evaluador de solución.
Consumo de Créditos	Pago único por libro (~\$0.75 USD).	Fraccional por problema resuelto (~\$0.005 a \$0.03 USD).
Ahorro Inteligente	Persistencia de corpus destilado (-usar-corpus).	Caché local de embeddings (embeddings.npz).

Tabla 5: Matriz comparativa integral de las Fases 1 y 2 del ecosistema de IA.

Recuerda

El enfoque neuro-simbólico desacopla la potencia de abstracción lingüística del LLM de la precisión matemática. Al forzar al modelo a emitir código SymPy verificado en sandbox, se eliminan por diseño las alucinaciones numéricas y se garantiza la reproducibilidad técnica exigida en ingeniería.